Project – Chat with Logistics Documents(RAG)

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s)

Sr No	Student Name	Enrolment Number
1	HATIM BOHRA	EN22CS301418
2	INDRA KAMAL PATEL	EN22CS301434
3	GOUTAM JAT	EN22CS301371
4	ISHIKA AGRAWAL	EN22CS301442
5	HARSH SOLANKI	EN22CS301394
6	HITESH AHUJA	EN22CS301426

Group Name: Group 09,D4

Project Number: GAI-45

Industry Mentor Name: Mr. Suraj Nayak

University Mentor Name: Prof. Divya Kumawat

Academic Year: 2025-26

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.
2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.
3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects
8. References

1. Introduction

1.1 Scope of the Document

This High Level Design (HLD) document describes the architecture and design of the Logistics RAG System — a Retrieval-Augmented Generation platform powered by Google Gemini. The document covers the overall system architecture, component interactions, data flow, API design, data management strategies, security considerations, and performance requirements. It serves as the primary technical reference for development, integration, and future maintenance of the system.

1.2 Intended Audience

This document is intended for the following stakeholders:

1. Development Team: Engineers responsible for building and maintaining the system components.
2. University and Industry Mentors: For academic review and industry guidance.
3. Project Evaluators: For assessment of design quality and technical completeness.
4. QA Engineers: For understanding system behaviour and designing test strategies.

1.3 System Overview

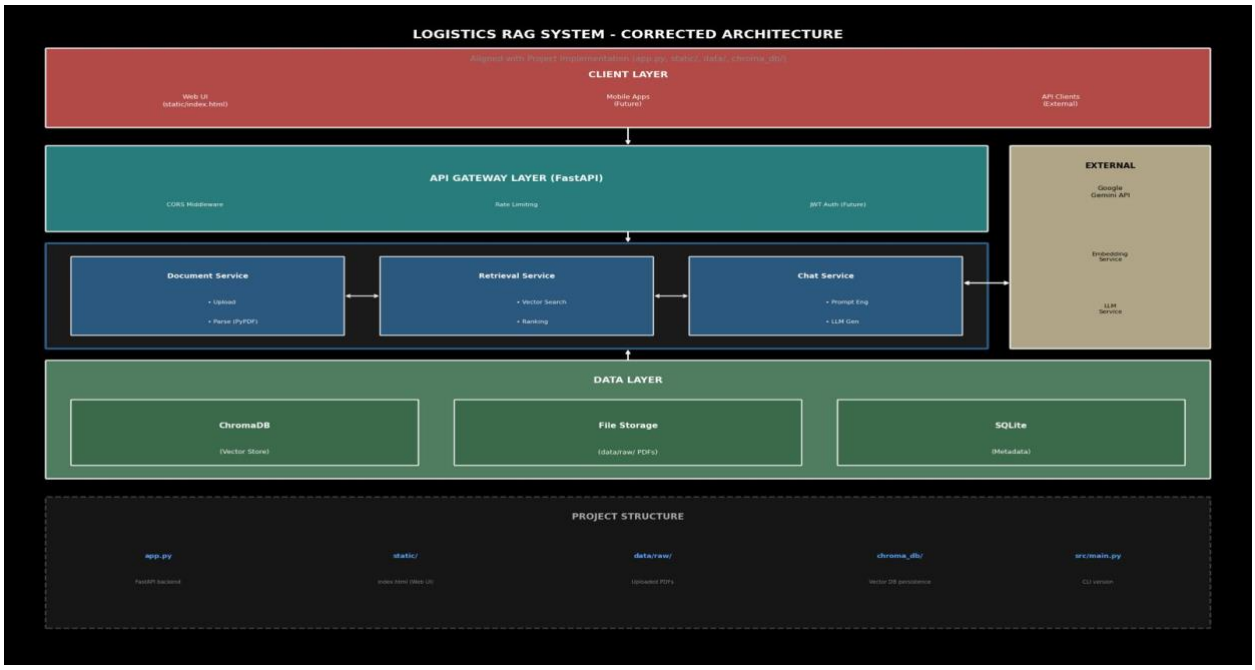
The Logistics RAG System is an AI-powered question-answering platform tailored exclusively for logistics and supply-chain domain documents. Users upload logistics-related PDFs through a Streamlit web interface; each PDF is validated by Gemini before ingestion. Accepted documents are chunked, embedded using Google Generative AI Embeddings, and stored in a ChromaDB vector database. Users can then ask natural-language questions through a chat interface, and the system retrieves the most relevant document excerpts and generates grounded answers with source citations (filename and page number). A FastAPI backend exposes all core operations as RESTful endpoints, while the Streamlit frontend provides a clean, theme-switchable UI.

2. System Design

2.1 Application Design

The system follows a two-tier client-server architecture:

5. Frontend Tier: A Streamlit application (streamlit_app.py) that communicates with the backend via HTTP REST calls. It manages session state, renders a chat interface, and handles PDF uploads through the sidebar.
6. Backend Tier: A FastAPI application (app.py) that orchestrates PDF validation, document ingestion, vector store management, and LLM-based answer generation. All AI operations use the Google Generative AI SDK (Gemini 2.5 Flash and Gemini Embedding 001).
7. Vector Store: ChromaDB persisted to the local filesystem (chroma_db/) serves as the retrieval layer. The embedding model converts document chunks into dense vectors for similarity search.

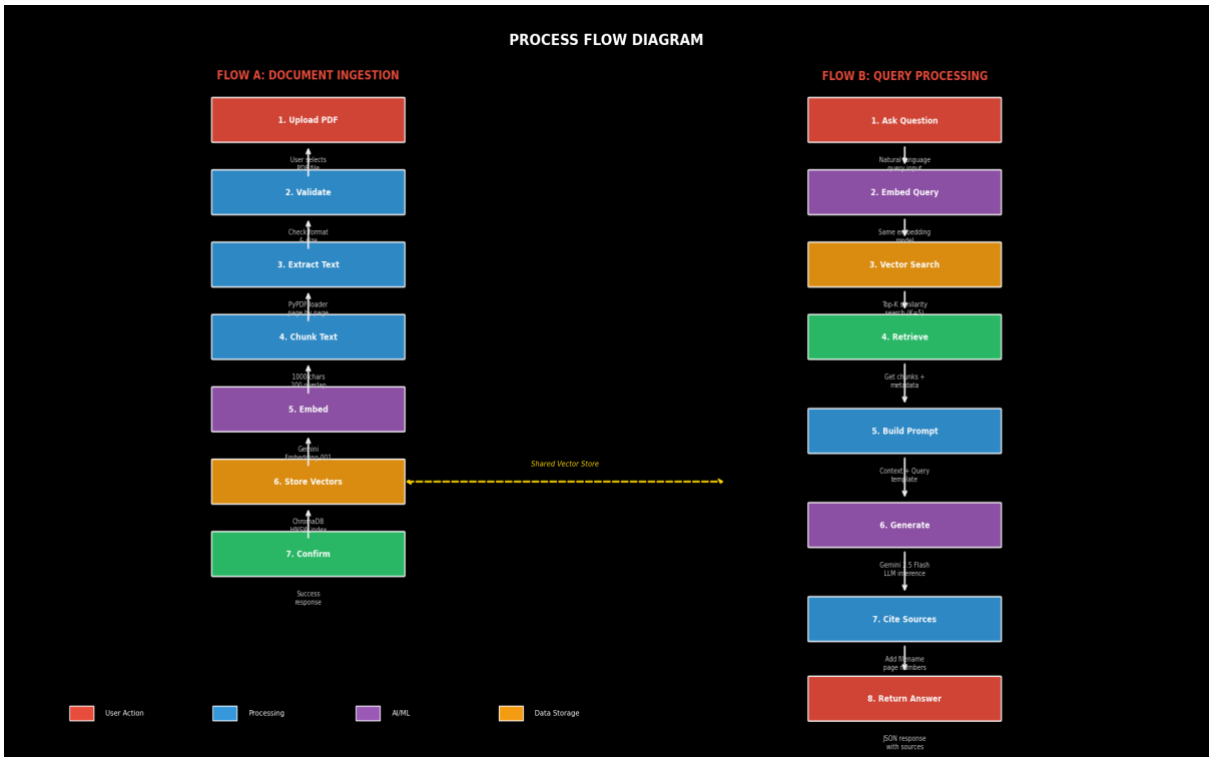


2.2 Process Flow

PDF Upload and Validation Flow: The user selects one or more PDF files in the Streamlit sidebar and clicks Upload. The frontend POSTs the files to the FastAPI /upload endpoint. The backend saves each file temporarily, extracts the first three pages of text, and sends a classification prompt to Gemini. If Gemini (or keyword heuristics) determines the document is logistics-related, the file is accepted. Otherwise it is deleted and the user is notified with the rejection reason.

Document Ingestion Flow: Accepted PDFs are loaded using PyPDFLoader and split into chunks of 1,000 tokens with 200-token overlap using RecursiveCharacterTextSplitter. Each chunk is tagged with its source filename and page number, then embedded and added to ChromaDB incrementally — no full rebuild is required when adding new documents.

Question Answering Flow: The user types a question in the chat input. The frontend POSTs it to /chat. The backend performs a similarity search (top-5 chunks) against ChromaDB, assembles a context string with chunk excerpts and their metadata, and sends a structured prompt to Gemini 2.5 Flash. The model returns an answer grounded strictly in the retrieved excerpts. The response includes the answer text and a list of source citations.



2.3 Information Flow

User to Streamlit UI to FastAPI Backend to Gemini LLM / ChromaDB to FastAPI Backend to Streamlit UI to User. All information exchange between Streamlit and FastAPI uses JSON over HTTP. The vector store is accessed locally via the ChromaDB Python client. The Gemini API is accessed over HTTPS using the langchain-google-genai integration and the GOOGLE_API_KEY environment variable.

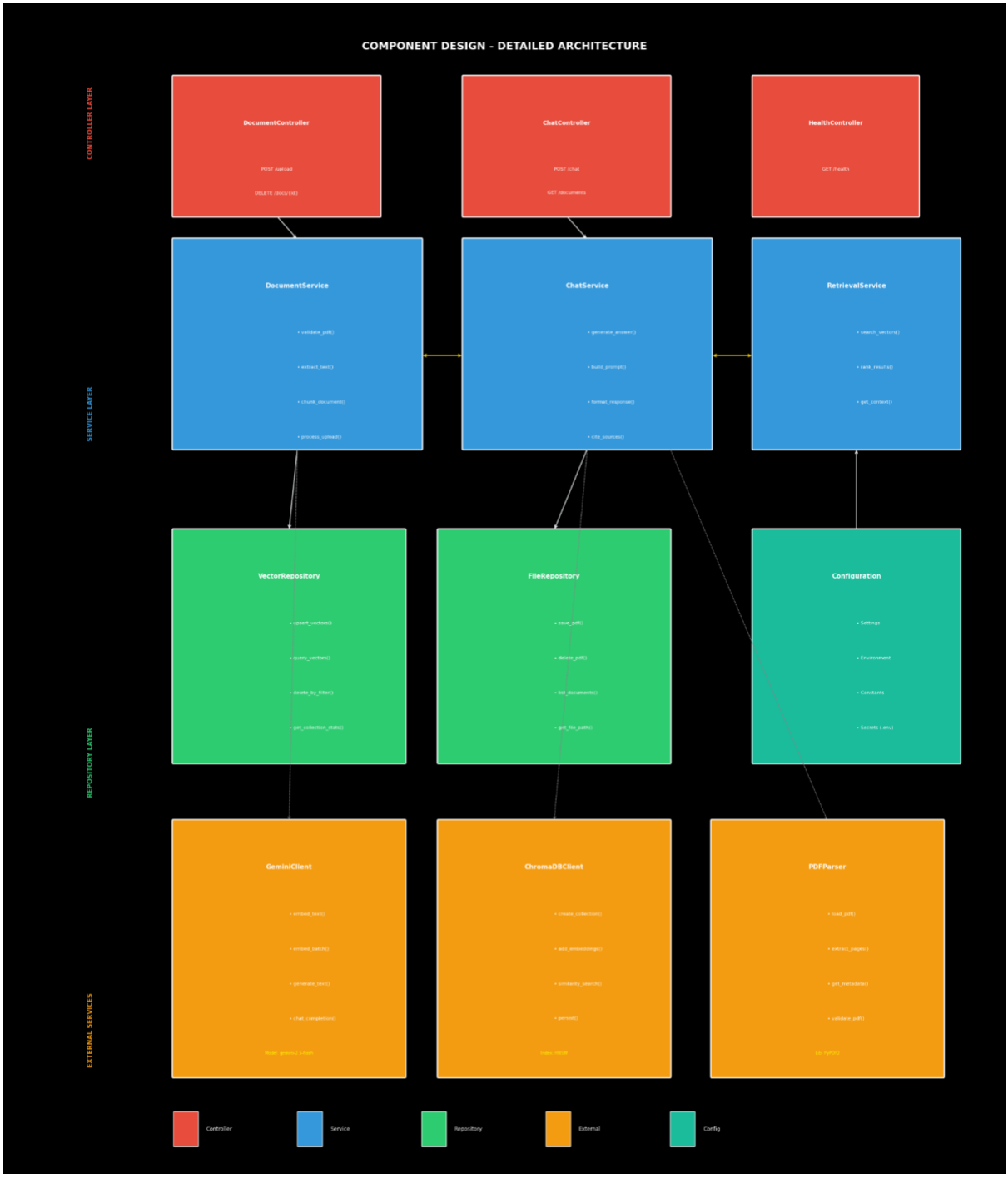
2.4 Components Design

app.py (FastAPI Backend): Implements all REST endpoints, orchestrates validation and ingestion, manages global vectorstore state, and invokes Gemini for classification and answer generation.

streamlit_app.py (Streamlit Frontend): Manages UI state (dark/light mode, chat history, document list). Communicates with the backend via the requests library. Renders chat bubbles, source citation cards, and upload progress.

ChromaDB: Persistent vector store stored in chroma_db/. Collection name: logistics_docs. Initialized on startup from disk if present; rebuilt or incrementally updated on document changes.

Google Gemini Integration: Uses models/gemini-2.5-flash for both document classification and answer generation, and models/gemini-embedding-001 for document chunk embeddings. Temperature is set to 0 for deterministic outputs.



2.5 Key Design Considerations

8. Logistics-Only Guardrail: A two-pass validation (keyword heuristics + Gemini classification) ensures only domain-relevant documents enter the knowledge base, preventing off-topic answers.
9. Incremental Ingestion: New PDFs are added to ChromaDB without rebuilding the entire index, reducing latency for multi-document workflows.

10. Source Attribution: Every answer includes filename and page number references, enabling users to verify information in the original documents.
11. Stateless Backend: The FastAPI server maintains only a single global vectorstore reference; all user session state is managed client-side in Streamlit.

2.6 API Catalogue

POST /upload: Accepts one or multiple PDF files. Validates each as a logistics document. Returns a summary of accepted and rejected files with chunk counts.

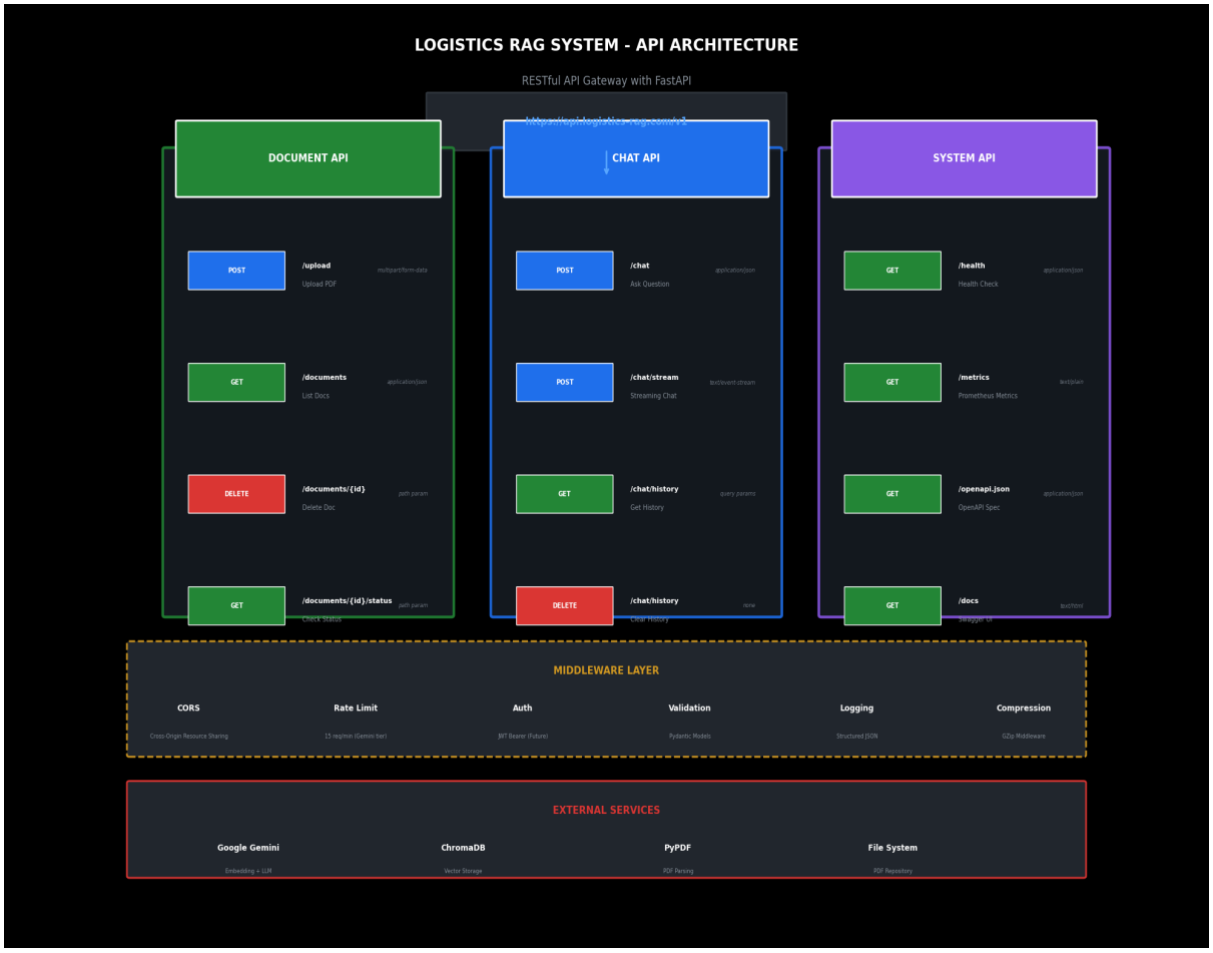
POST /chat: Accepts a JSON body with question (string) and include_sources (boolean). Returns the generated answer and a list of source citations.

GET /documents: Returns a list of all uploaded PDF documents with filenames and sizes.

DELETE /documents/{filename}: Deletes a specific document and rebuilds the vector store from remaining documents.

GET /health: Returns backend health status and whether the vector store is initialized.

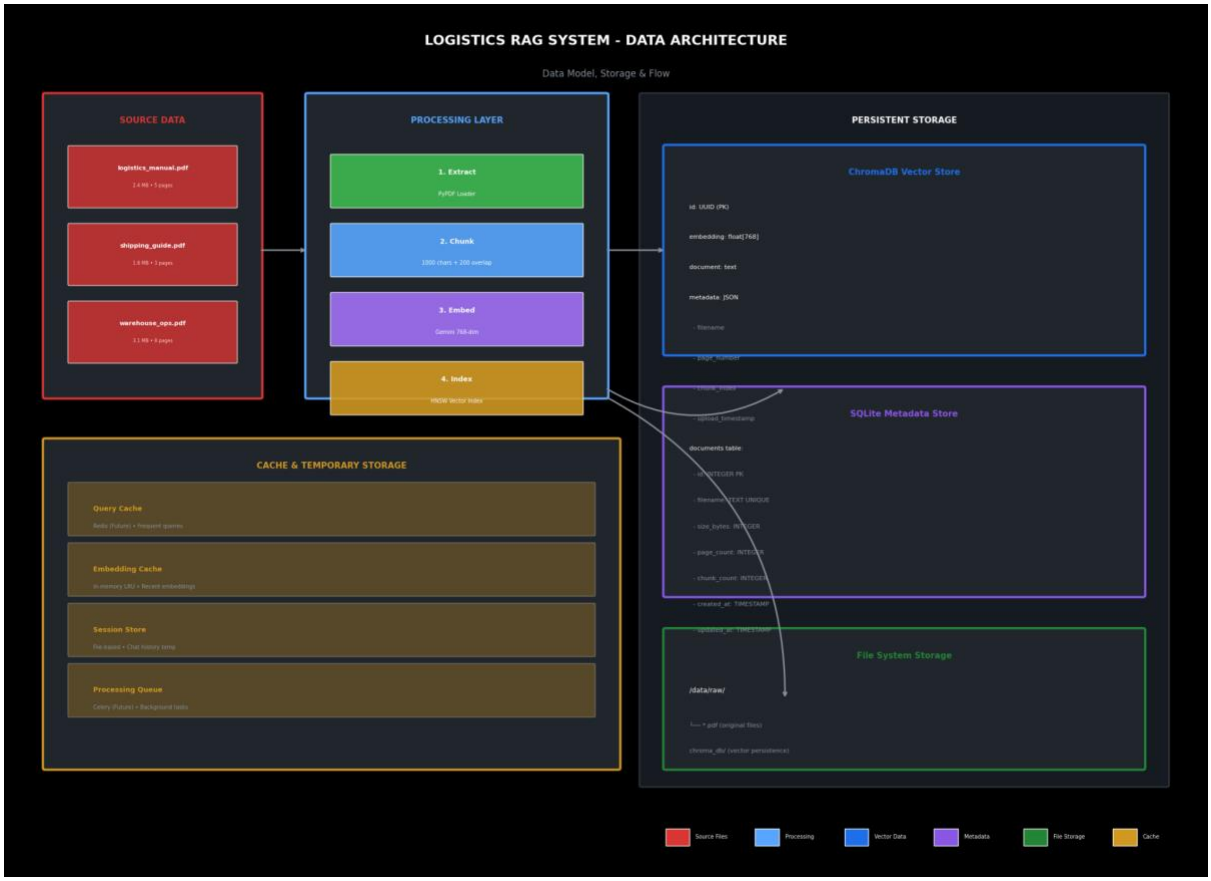
GET /: Serves the legacy static HTML frontend.


LOGISTICS RAG SYSTEM - API ARCHITECTURE
RESTful API Gateway with FastAPI
DOCUMENT API
POST
/upload
Upload PDF
multipart/form-data
GET
/documents
List Docs
application/json
DELETE
/documents/{id}
Delete Doc
path param
GET
/documents/{id}/status
Check Status
path param
CHAT API
POST
/chat
Ask Question
application/json
POST
/chat/stream
Streaming Chat
text/event-stream
GET
/chat/history
Get History
query params
DELETE
/chat/history
Clear History
none
SYSTEM API
GET
/health
Health Check
application/json
GET
/metrics
Prometheus Metrics
text/plain
GET
/openapi.json
OpenAPI Spec
application/json
GET
/docs
Swagger UI
text/html
MIDDLEWARE LAYER
CORS
Cross-Origin Resource Sharing
Rate Limit
15 req/min (Gemini tier)
Auth
JWT Bearer (Future)
Validation
Pydantic Models
Logging
Structured JSON
Compression
GZip Middleware
EXTERNAL SERVICES
Google Gemini
Embedding + LLM
ChromaDB
Vector Storage
PyPDF
PDF Parsing
File System
PDF Repository

3. Data Design

3.1 Data Model

The system operates with two primary data entities: raw PDF files stored on the filesystem under data/raw/, and document chunks stored as vector embeddings in ChromaDB. Each chunk record contains the text content, the source filename, the page number, and the dense embedding vector produced by Gemini Embedding 001. No relational database is used; the vector store is the sole persistent data layer.



3.2 Data Access Mechanism

Document chunks are retrieved via ChromaDB similarity search using the cosine distance metric. The top-K (default K=5) most similar chunks to the user query are returned and assembled into a context window for the LLM. Direct file access is used for PDF storage and deletion. The Chroma client is initialized at startup and shared as a global singleton throughout the FastAPI application lifetime.

3.3 Data Retention Policies

All uploaded PDFs are retained indefinitely in data/raw/ until explicitly deleted via the DELETE /documents/{filename} API endpoint. ChromaDB vectors persist on disk in chroma_db/ and survive server restarts. When a document is deleted, the entire vector

store is rebuilt from the remaining PDFs to maintain index consistency. No automated expiry or archival policies are implemented in the current version.

3.4 Data Migration

No formal data migration mechanism exists in the current version. If the embedding model changes or the ChromaDB schema evolves, the recommended procedure is to delete the chroma_db/ directory and re-upload all source PDFs to trigger a full re-ingestion. The raw PDFs in data/raw/ serve as the source of truth and enable full reconstruction of the vector index at any time.

4. Interfaces

External Interfaces: The system exposes a REST API over HTTP (default port 8000) for all backend operations. The Streamlit frontend communicates exclusively through this API and is accessed by end users via a web browser at port 8501.

Google Gemini API: The backend communicates with Google's Generative AI API over HTTPS. Authentication is handled via the GOOGLE_API_KEY environment variable loaded from a .env file.

ChromaDB Interface: ChromaDB is accessed via its Python client library using a local persist directory. No network interface is required; it operates as an embedded database within the FastAPI process.

User Interface: The Streamlit UI provides a sidebar for PDF upload and document management, a main chat area for question-answering, a dark/light mode toggle, and collapsible source citation cards beneath each answer.

5. State and Session Management

Session State (Frontend): Streamlit's st.session_state dictionary is used to persist the following within a browser session: chat_history (list of message dicts), dark_mode (boolean), docs_ready (boolean indicating whether documents are loaded), and ingest_log (list of ingestion status messages). This state is not persisted across browser refreshes.

Application State (Backend): The FastAPI backend maintains a single global Python variable vectorstore that holds the ChromaDB client reference. This is initialized at startup from the persisted chroma_db/ directory if it exists, and updated in-place when documents are added or deleted. Because it is a module-level global, it is shared across all requests within the same process.

Stateless Request Handling: Each API request is otherwise stateless. The complete context required to answer a question (the query text) is provided in the request body; no server-side session cookies or tokens are used.

6. Caching

Vector Store Persistence: ChromaDB persists embeddings to disk, which acts as a long-term cache of pre-computed document embeddings. Re-uploading the same PDF will result in duplicate chunks unless the file is explicitly deleted first.

In-Memory Vector Store Reference: The global vectorstore object is held in memory for the lifetime of the FastAPI process, avoiding repeated disk reads for every query. On server restart, ChromaDB is reloaded from the persist directory in the startup event handler.

No HTTP-Level Caching: The current implementation does not implement HTTP response caching, Redis, or any other distributed cache. All LLM calls to Gemini are made per request without response memoization. Future versions may introduce query-level caching to reduce API costs for repeated questions.

7. Non-Functional Requirements

7.1 Security Aspects

12. API Key Management: The GOOGLE_API_KEY is stored in a .env file and loaded at runtime via python-dotenv. The key is never exposed to the frontend or included in API responses.
13. CORS Policy: The FastAPI backend is configured with allow_origins=['*'] for development. In production, this should be restricted to the specific Streamlit frontend origin.
14. Input Validation: File uploads are validated for the .pdf extension. Document content is classified by Gemini before ingestion, preventing arbitrary content injection into the knowledge base.
15. No Authentication: The current version does not implement user authentication or authorization. All API endpoints are publicly accessible. Production deployments should add token-based authentication.

7.2 Performance Aspects

16. Chunk Size Tuning: Chunks of 1,000 tokens with 200-token overlap balance retrieval precision with context richness, avoiding overly fragmented or overly large context windows.
17. Top-K Retrieval: Only the top 5 most relevant chunks are passed to the LLM, keeping the prompt context manageable and reducing token costs.
18. Incremental Ingestion: New documents are added to ChromaDB without a full index rebuild, keeping upload latency proportional to document size rather than total corpus size.
19. Scalability Constraints: The single-process, single-global-state design is suitable for small teams. For production scale, a horizontally scalable vector database and an async task queue for ingestion would be required.

8. References

20. Google Generative AI SDK Documentation — https://ai.google.dev/gemini-api/docs
21. LangChain Documentation — https://python.langchain.com/docs
22. ChromaDB Documentation — https://docs.trychroma.com
23. FastAPI Documentation — https://fastapi.tiangolo.com
24. Streamlit Documentation — https://docs.streamlit.io
25. PyPDF Documentation — https://pypdf.readthedocs.io
26. Project Repository — logistics-rag-main (submitted as part of GAI-45, Group 09D4)